Open in Colab

Functional Programming

In Python functional programming means using functions as arguments to other functions. Some functions that uses other functions as atguments are:

Anonymuous Function

Another way of defining a function in Python is `lambda` expressions. In `lambda` function definition there must be only one expression (may be compound). No more expressions and no statements at all. Since the statements are not allowed inside `lambda` function, thre is no `return` statement. It returns the result of that unique expression.

In [68]:
```
factorial = lambda x: x*factorial(x-1) if x != 0 else 1
```

`map` Function

`map` function takes two arguments, first is a function and second is an iterable. And applies the function to each item of the iterable.

In [69]:
```
for i in map(factorial, range(10, 16)):
    print(i)
```

```
3628800
39916800
479001600
6227020800
87178291200
1307674368000
```

This is equivalent to:

In [70]:
```
for i in range(10, 16):
    print(factorial(i))
```

```
3628800
39916800
479001600
6227020800
87178291200
1307674368000
```

Since `lambda` functions are expressions they can be defined inside `map` functions.

In [71]:
```
for i in map(lambda x:x**3, range(10, 16)):
    print(i)
```

```
1000
1331
1728
2197
2744
3375
```

`zip()` Function

`zip()` function takes two iterables and generates a sequence of tuples. Fist item from the first iterable and second item from the second iterable.

In [72]:
```
dict(kv for kv in zip(range(0, 10, 1), range(0, 20, 2)))
```

Out[72]:
```
{0: 0, 1: 2, 2: 4, 3: 6, 4: 8, 5: 10, 6: 12, 7: 14, 8: 16, 9: 18}
```

is like:

In [73]:
```
iterable1 = tuple(range(0, 10, 1))
iterable2 = tuple(range(0, 20, 2))
dict((iterable1[i], iterable2[i])
     for i in range(min(len(iterable1), len(iterable1))))
```

Out[73]:
```
{0: 0, 1: 2, 2: 4, 3: 6, 4: 8, 5: 10, 6: 12, 7: 14, 8: 16, 9: 18}
```

`filter` Function

`filter` function selects some items from an iterable according to some function result.

In [74]:
```
print(*filter(lambda x: x & 1, range(15)))
```

```
1 3 5 7 9 11 13
```

In [75]:
```
sum(map(lambda x: x**2, filter(lambda x: x & 1, range(10000))))
```

Out[75]:
```
166666665000
```

This is equivalent to:

In [76]:
```
sum(i**2 for i in range(10000) if i & 1)
```

Out[76]:
```
166666665000
```

`all()` & `any()` Functions

Builtin `all` and `any` functions can be used to test the truth of tuples, lists, iterables.

In [77]:
```
all((1 == 1., 1 < 2, 4 > 3, 3 != 4))
```

Out[77]:
```
True
```

In [78]:
```
all((0, 1, 2, 3))
```

Out[78]:
```
False
```

The function `all` is equivalent to:

In [79]:
```
def all(iterable):
    for element in iterable:
        if not element:
            return False
    return True
```

In [80]:
```
any((1 == 1., 1 > 2, 4 < 3, 3 == 4))
```

Out[80]:
```
True
```

In [81]:
```
any((0, 0.0, "", (), [], {}))
```

Out[81]:
```
False
```

The function `any` is equivalent to:

In [82]:
```
def any(iterable):
    for element in iterable:
        if element:
            return True
    return False
```

Nested Functions

Functions in Python can be nested i.e You can define a function inside another function definition. Since functions are some Python objects they can be returned by or passed as arguments to other functions.

In [83]:
```
def adder(a):
  "the function adder here defines another function which adds `a` to any given numbe
  "by doing this you can partially apply addition on numbers"
  def adder_a(b):
    return a+b
  return adder_a
```

In [84]:
```
f1 = adder(2)
f1(3)
```

Out[84]:
```
5
```

or equivalently:

In [85]:
```
adder(2)(3)
```

Out[85]:
```
5
```

Decoraters

Decoraters are special nested functions that convert a function to another function.

The following function checks the arguments of a function before executing, and prints an error message if any of them is negative. This can be used in our implementation of `gcd` function to prevent infinite loops.

In [86]:
```
def ensure_positive(func):
    def wrapper(*xs):
        if any(x < 0 for x in xs):
            print("ERROR: Arguments must be non-negative")
            return
        return func(*xs)
    return wrapper
```

To apply a decorator to a function use `@` and decorator name before `def` statement line.

In [87]:
```
@ensure_positive
```

```
def gcd(a,b):
  "this function calculates the GCD of given numbers in ancient way."
  while a != b:
    if a > b:
      a -= b
    if a < b:
      b -= a

  return a
```

Noe `gcd` function is safer.

In [88]:
```
gcd(-2,12)
```

```
ERROR: Arguments must be non-negative
```

Similarly we can apply `ensure_positive` decorator to `factorial` function as well:

In [89]:
```
@ensure_positive
def factorial(n):
  "recursive factorial function"
  if n == 0:
    return 1
  return n * factorial(n-1)
```

In [90]:
```
factorial(-5)
```

```
ERROR: Arguments must be non-negative
```